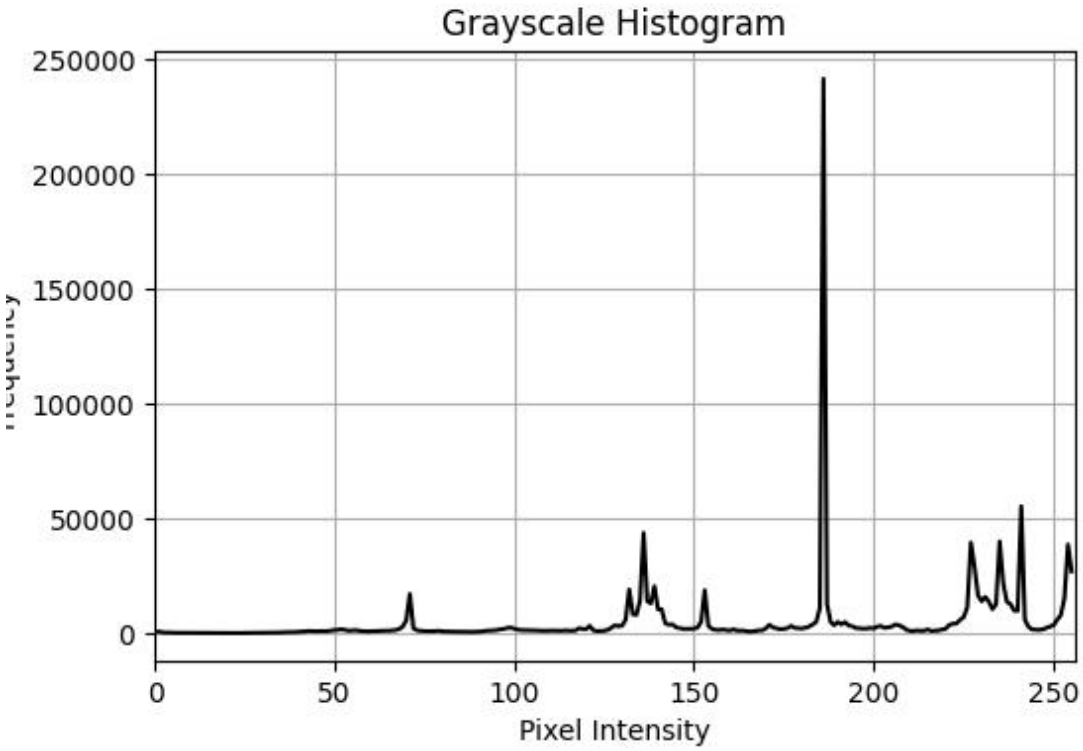
HW 6

Problem 1

原图

灰度图的直方图



Problem 2

进行Histogram Equalization后得到的图像是

Problem 3

经过四种不同直方图变换得到的图片分别为

Problem 4

经过Image Histogram Matching得到的图片是

Code

```
import cv2
import numpy as np
import os
import matplotlib.pyplot as plt

output_dir = "output_images"
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

def save_histogram(image_path, output_path):
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if image is None:
        print(f"Error: Unable to read {image_path}")
        return

    hist = cv2.calcHist([image], [0], None, [256], [0, 256])
```

```
    plt.savefig(output_path)
    plt.close()
    print(f"Histogram saved to {output_path}")

def histogram_equalization(image_path, output_path):
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if image is None:
        print(f"Error: Unable to read {image_path}")
        return

    equalized = cv2.equalizeHist(image)
    cv2.imwrite(output_path, equalized)
    print(f"Equalized image saved to {output_path}")

def histogram_matching(source_image, target_image):
    hist_source, _ = np.histogram(source_image.flatten(), bins=256, range=[0,
256], density=True)
    hist_target, _ = np.histogram(target_image.flatten(), bins=256, range=[0,
256], density=True)

    cdf_source = hist_source.cumsum()
    cdf_target = hist_target.cumsum()

    cdf_source = (cdf_source / cdf_source[-1]) * 255
    cdf_target = (cdf_target / cdf_target[-1]) * 255

    if len(cdf_target) != 256:
        cdf_target = np.pad(cdf_target, (0, 256 - len(cdf_target)), mode='edge')

    lookup_table = np.interp(np.arange(256), cdf_source, cdf_target)
    matched_image = cv2.LUT(source_image, lookup_table.astype(np.uint8))

    return matched_image

def apply_four_histograms(image_path):
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if image is None:
        print(f"Error: Unable to read {image_path}")
        return

    height, width = image.shape
    random_target_1 = np.random.randint(0, 256, (height, width), dtype=np.uint8)
    random_target_2 = np.linspace(0, 255, num=height * width,
dtype=np.uint8).reshape(height, width)
    random_target_3 = np.full((height, width), 128, dtype=np.uint8)
    random_target_4 = cv2.GaussianBlur(image, (21, 21), 0)

    matched_1 = histogram_matching(image, random_target_1)
    matched_2 = histogram_matching(image, random_target_2)
    matched_3 = histogram_matching(image, random_target_3)
    matched_4 = histogram_matching(image, random_target_4)

    cv2.imwrite(os.path.join(output_dir, "matched_1.jpg"), matched_1)
    cv2.imwrite(os.path.join(output_dir, "matched_2.jpg"), matched_2)
```

```
    cv2.imwrite(os.path.join(output_dir, "matched_3.jpg"), matched_3)
    cv2.imwrite(os.path.join(output_dir, "matched_4.jpg"), matched_4)
    print("Four transformed images saved.")

def histogram_matching_color(source_image_path, target_image_path, output_path):
    source_img = cv2.imread(source_image_path)
    reference_img = cv2.imread(target_image_path)

    if source_img is None or reference_img is None:
        print("Error: Unable to read one or both images.")
        return

    matched_channels = []
    for i in range(3):  # BGR 通道
        matched_channel = histogram_matching(source_img[:, :, i],
reference_img[:, :, i])
        matched_channels.append(matched_channel)

    matched_color_img = cv2.merge(matched_channels)
    cv2.imwrite(output_path, matched_color_img)
    print(f"Color histogram matched image saved to {output_path}")

# ================================

input_gray_image = "1.jpg"
input_color_image = "1.jpg"
reference_color_image = "2.jpg"

save_histogram(input_gray_image, os.path.join(output_dir, "histogram.jpg"))

histogram_equalization(input_gray_image, os.path.join(output_dir,
"equalized.jpg"))

apply_four_histograms(input_gray_image)

histogram_matching_color(input_color_image, reference_color_image,
os.path.join(output_dir, "matched_color.jpg"))
```